МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

ЛАБОРАТОРНАЯ РАБОТА №5

по дисциплине

«Проектирование вычислительных систем»

Вариант: 5

Работу выполнили:
Студенты группы P3411
Болорболд Аригуун
Кравцов Кирилл Денисович

Преподаватель:
Пинкевич Василий Юрьевич

г. Санкт-Петербург
2025

Содержание

1. Текст задания 3
2. Выполнение 4
   2.1. Распиновка 4
   2.2. Конфигурация частоты ядра 4
   2.3. Блок-схема: 5
   2.4. Описание работы алгоритма 5
   2.5. Код драйвера 5
3. Вывод 9

1. Текст задания

Разработать программу, которая включает драйвер OLED-дисплея. Для организации обмена данными с дисплеем можно использовать те же три способа, что и в предыдущей работе. Поскольку дисплей управляется по той же шине I*2*C, что и клавиатура, необходимо избежать конфликтов доступа к шине. При работе с I^2C по опросу они не возникнут, а при работе по прерыванию следует планировать вычисления так, чтобы транзакции не пересекались по времени.

Если транзакции I^2C генерируются по прерыванию от таймера, следует встроить транзакции обновления видеопамяти дисплея в расписание опроса клавиатуры. При этом следует учесть, что отправка буфера данных в контроллер дисплея происходит значительно дольше, чем любая транзакция опроса клавиатуры. Например, после восьми вызовов обработчика прерываний для опроса клавиатуры можно инициировать отправку буфера и несколько вызовов обработчика отправлять никаких транзакций по I^2C, давая время на завершение отправки буфера. Начальную инициализацию дисплея, как и клавиатуры, удобнее делать в режиме опроса, пока не включены прерывания от таймера.

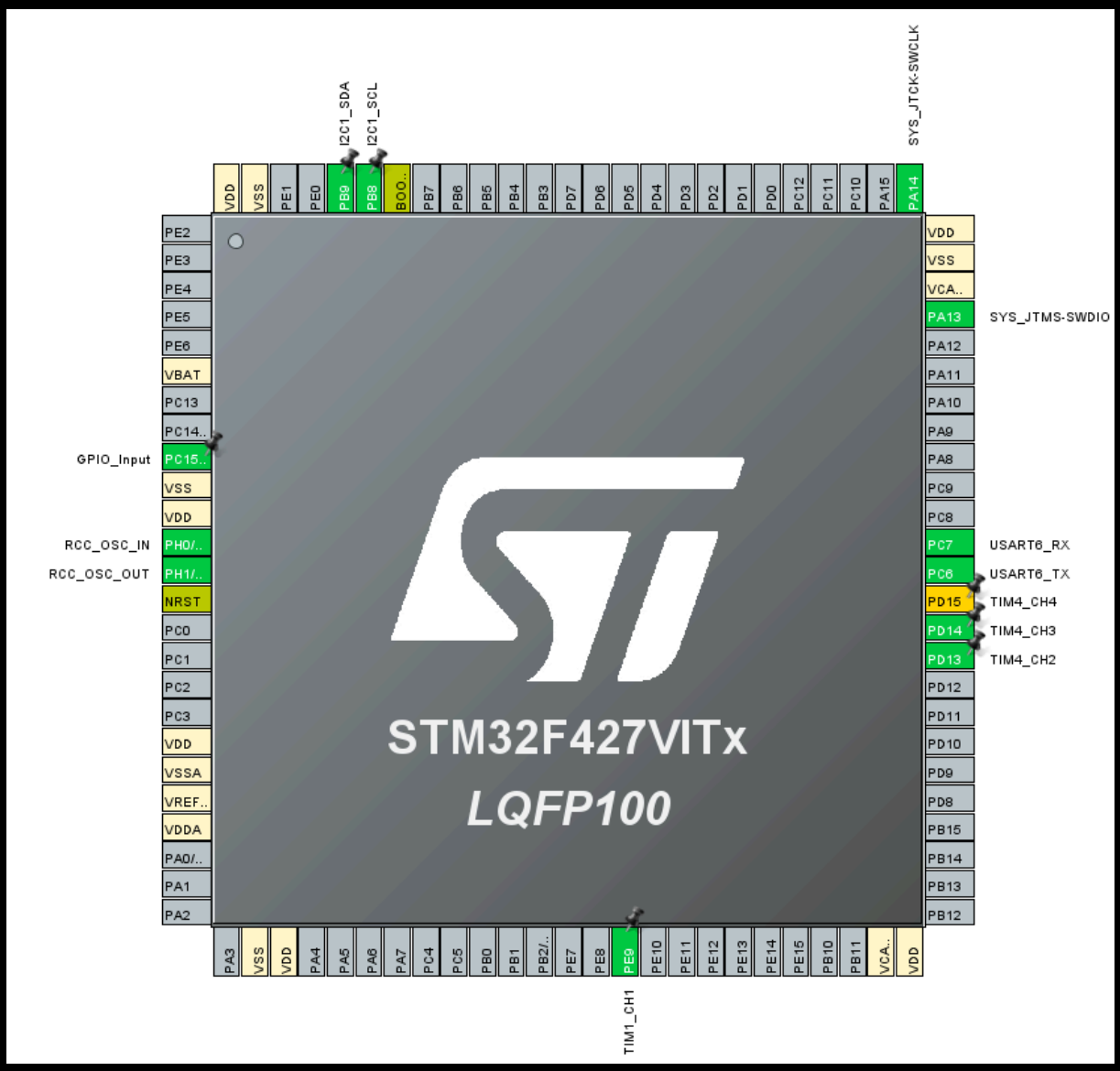
Адаптировать программу-кодовый замок для использования с дисплеем и клавиатурой `SDK-1.1M`. Кнопки клавиатуры использовать для:

- ввода пароля (цифры 0 – 9);
- перехода в режим изменения пароля/сохранения нового пароля.

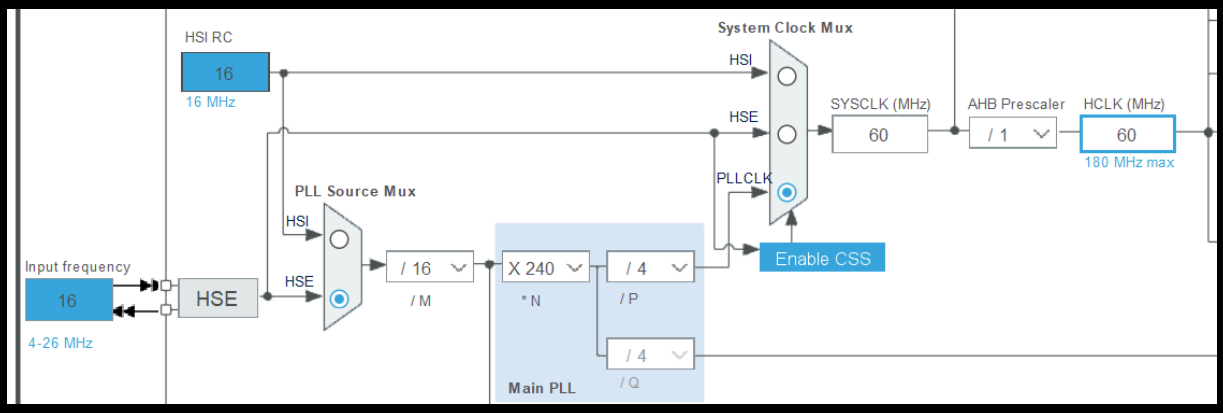
Отображение статуса выполняется на дисплее. В обычном режиме на экране выведено текстовое приглашение для ввода пароля. После ввода производится проверка его правильности, и на экран выводится соответствующее сообщение. При переходе в режим изменения пароля выводится текстовое приглашение вида «введите новый пароль». Длина пароля – от 8 до 12 цифр. Если пользователь пытается сохранить пароль неправильной длины, выводится сообщение об ошибке.

2. Выполнение

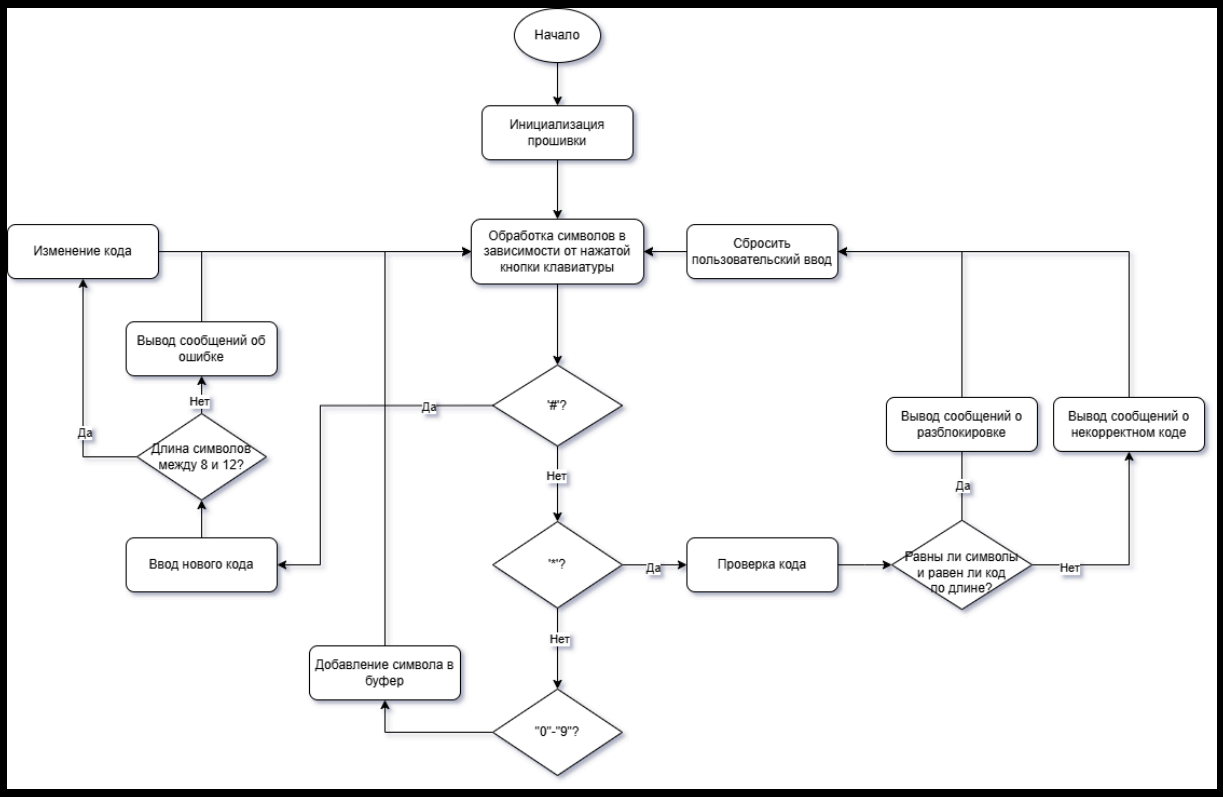
2.1. Распиновка



2.2. Конфигурация частоты ядра



2.3. Блок-схема:



2.4. Описание работы алгоритма

2.5. Код драйвера

```
#include <stdbool.h>

#include "main.h"
#include "kb.h"
#include "lcd.h"
#include "lock.h"

I2C_HandleTypeDef hi2c1;
TIM_HandleTypeDef htim6;

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C1_Init(void);
static void MX_TIM6_Init(void);

int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_I2C1_Init();
  MX_TIM6_Init();
```

```
  kb_init(&hi2c1);
  lcd_init(&hi2c1);
  HAL_TIM_Base_Start_IT(&htim6);
  lock_init(&hi2c1);

  static const char keymap[12] = {
      '1','2','3',
      '4','5','6',
      '7','8','9',
      '*','0','#'
  };
  static uint32_t last_key_time[12] = {0};
  const uint32_t KEY_DEDUPE_MS = 200;

  while (1) {
    if (kb_event_has()) {
        struct kb_event evt = kb_event_pop();
        if (evt.type == KB_EVENT_TYPE_PRESS) {
            uint32_t now = HAL_GetTick();
            if ((uint32_t)(now - last_key_time[evt.key]) >=
KEY_DEDUPE_MS) {
                last_key_time[evt.key] = now;

                char key = keymap[evt.key];
                lock_key_pressed(key);
            } else {
            }
        }
    }
  }
}

void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  __HAL_RCC_PWR_CLK_ENABLE();
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLM = 15;
  RCC_OscInitStruct.PLL.PLLN = 216;
```

```
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
  RCC_OscInitStruct.PLL.PLLQ = 4;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_PWREx_EnableOverDrive() != HAL_OK)
  {
    Error_Handler();
  }

  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|
RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|
RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5) !=
HAL_OK)
  {
    Error_Handler();
  }
}

static void MX_I2C1_Init(void)
{
  hi2c1.Instance = I2C1;
  hi2c1.Init.ClockSpeed = 400000;
  hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
  hi2c1.Init.OwnAddress1 = 0;
  hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
  hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
  hi2c1.Init.OwnAddress2 = 0;
  hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
  hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
  if (HAL_I2C_Init(&hi2c1) != HAL_OK)
  {
    Error_Handler();
  }

  if (HAL_I2CEx_ConfigAnalogFilter(&hi2c1,
I2C_ANALOGFILTER_ENABLE) != HAL_OK)
  {
```

```
    Error_Handler();
  }

  if (HAL_I2CEx_ConfigDigitalFilter(&hi2c1, 0) != HAL_OK)
  {
    Error_Handler();
  }
}

static void MX_TIM6_Init(void)
{
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  htim6.Instance = TIM6;
  htim6.Init.Prescaler = 8999;
  htim6.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim6.Init.Period = 9;
  htim6.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_Base_Init(&htim6) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim6,
&sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }

}

static void MX_GPIO_Init(void)
{

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef * htim) {
    if (htim->Instance == TIM6) {
        static bool lcd = false;

        if (HAL_I2C_GetState(&hi2c1) != HAL_I2C_STATE_READY) {
            return;
```

```
        }

        if (lcd) {
            lcd_step(&hi2c1);
        } else {
            kb_scan_step(&hi2c1);
        }

        lcd = !lcd;
    }
}
```

3. Вывод

В ходе выполнения работы мы ознакомились с принципом работы OLED-экрана. Экран работает по принципу выведения данных, полученный через I^2C с предварительным хранением в буфере, а данные выводится с помощью драйвера SSD1306 или его аналога. Эта работа являлась справкой с тем, как внутренне устроены визуальные электронные техники.